

✓ ASMIT SAHU

23052231

Objective: Introduce deep learning basics with neural networks.

Experiment: Build a simple neural network to classify handwritten digits using TensorFlow/Keras.

- The images are flattened from 28x28 matrices into 1D vectors of length 784 to be fed into the neural network.
- Input Layer: The input is a flattened vector of size 784 (28x28).
- Hidden Layer: A dense layer with 128 neurons and ReLU activation. This layer allows the model to learn complex patterns.
- Dropout Layer: A dropout layer with a rate of 0.2, which helps reduce overfitting by randomly setting some of the weights to zero during training.
- Output Layer: A softmax layer with 10 neurons for classifying the digits from 0 to 9.
- Show the Test Accuracy, model's performance over epochs(accuracy plot), loss plot indicates whether the model is converging Note: We can use MNIST Dataset: The MNIST dataset contains 60,000 training images and 10,000 test images of handwritten digits (0-9)

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])
```

```

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                   epochs=10,
                   batch_size=32,
                   validation_data=(x_test, y_test))

test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"\n✅ Test Accuracy: {test_accuracy * 100:.2f}%")

```

```

Epoch 1/10
1875/1875 ————— 12s 6ms/step - accuracy: 0.8564 - loss: 0.4923 -
Epoch 2/10
1875/1875 ————— 10s 6ms/step - accuracy: 0.9537 - loss: 0.1528 -
Epoch 3/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9659 - loss: 0.1102 -
Epoch 4/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9728 - loss: 0.0889 -
Epoch 5/10
1875/1875 ————— 9s 5ms/step - accuracy: 0.9764 - loss: 0.0756 - v
Epoch 6/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9777 - loss: 0.0684 -
Epoch 7/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9813 - loss: 0.0576 -
Epoch 8/10
1875/1875 ————— 9s 5ms/step - accuracy: 0.9831 - loss: 0.0517 - v
Epoch 9/10
1875/1875 ————— 9s 5ms/step - accuracy: 0.9846 - loss: 0.0473 - v
Epoch 10/10
1875/1875 ————— 10s 5ms/step - accuracy: 0.9869 - loss: 0.0415 -
313/313 ————— 1s 2ms/step - accuracy: 0.9747 - loss: 0.0873

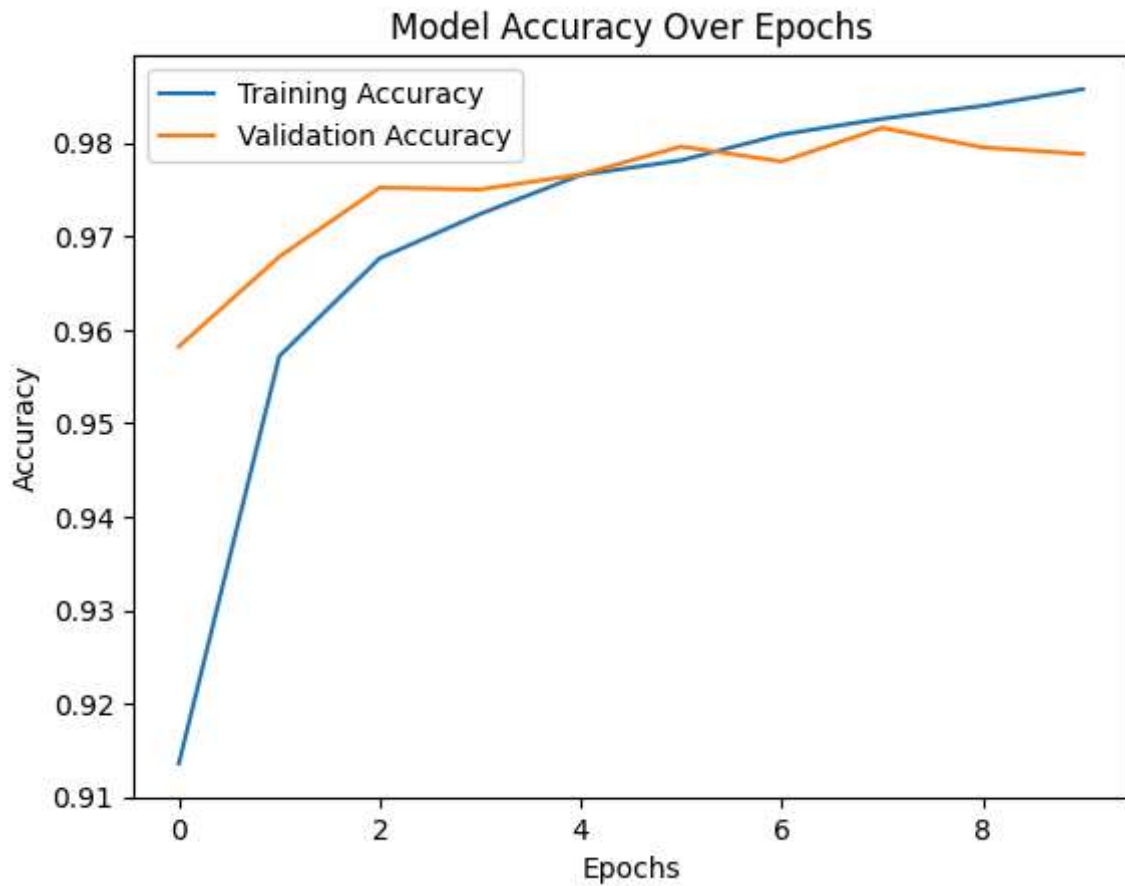
✅ Test Accuracy: 97.88%

```

```

plt.figure()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Model Accuracy Over Epochs')
plt.legend()
plt.show()

```



```
plt.figure()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Model Loss Over Epochs')
plt.legend()
plt.show()
```

